

Comenzado el	jueves, 25 de julio de 2024, 08:25
Estado	Finalizado
Finalizado en	jueves, 25 de julio de 2024, 08:54
Tiempo empleado	28 minutos 47 segundos
Puntos	15/20
Calificación	8 de 10 (76%)

Pregunta **1**

Parcialmente correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuáles de las siguientes propuestas generales son **ciertas** en relación al segmento de memoria conocido como **Stack Segment**?

Seleccione una o más de una:

☐ a.

El Stack Segment se utiliza como soporte interno en el proceso de invocación a funciones (**con o sin recursividad**): se va llenando a medida que se desarrolla la cascada de invocaciones, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.

☒ b.

El Stack Segment funciona como una pila (o apilamiento) de datos (modalidad **LIFO**: Last In - First Out): el último dato en llegar, se almacena en la cima del stack, y será por eso el primero en ser retirado.

¡Correcto!

☒ c.

El Stack Segment se utiliza como soporte interno **solamente** en el proceso de invocación a funciones recursivas: se va llenando a medida que la cascada recursiva se va desarrollando, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.

Incorrecto... la palabra **solamente** es lo que hace incorrecta esta afirmación...

☐ d.

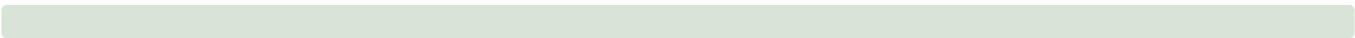
El Stack Segment funciona como una cola (o fila) de datos (modalidad **FIFO**: First In - First Out): el primer dato en llegar, se almacena al frente del stack, y será por eso el primero en ser retirado.

Ha seleccionado correctamente 1.

Las respuestas correctas son:

El Stack Segment funciona como una pila (o apilamiento) de datos (modalidad **LIFO**: Last In - First Out): el último dato en llegar, se almacena en la cima del stack, y será por eso el primero en ser retirado.,

El Stack Segment se utiliza como soporte interno en el proceso de invocación a funciones (**con o sin recursividad**): se va llenando a medida que se desarrolla la cascada de invocaciones, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.



Historial de respuestas				
Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:27:09	Guardada: El Stack Segment funciona como una pila (o apilamiento) de datos (modalidad LIFO : Last In - First Out): el último dato en llegar, se almacena en la cima del stack, y será por eso el primero en ser retirado. ; El Stack Segment se utiliza como soporte interno _solamente_ en el proceso de invocación a funciones recursivas: se va llenando a medida que la cascada recursiva se va desarrollando, y se vacía a medida que se produce el proceso de regreso o vuelta atrás.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Parcialmente correcta	1

Pregunta **2**

Parcialmente correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuáles de las siguientes afirmaciones **son ciertas** en relación a conceptos asociados con la **recursividad**?

Seleccione una o más de una:

☒ a. A medida que se desarrolla la cascada de invocaciones recursivas, el stack segment se va llenando para darle soporte a cada instancia recursiva, y luego, cuando una instancia logra finalizar y se produce el proceso de vuelta atrás, el stack segment comienza a vaciarse. ¡Correcto!

☐ b. Cada instancia recursiva que es ejecutada, almacena dos grupos de datos en el stack segment: la dirección de retorno (a la que se debe regresar cuando termine la ejecución de esa instancia) y las variables locales que esa instancia de la función haya creado.

☐ c. Una función recursiva no puede incluir más de una invocación a si misma en su bloque de acciones.

☐ d. Si una función es recursiva, entonces no debe incluir ningún ciclo en su bloque de acciones.

Las respuestas correctas son:

Cada instancia recursiva que es ejecutada, almacena dos grupos de datos en el stack segment: la dirección de retorno (a la que se debe regresar cuando termine la ejecución de esa instancia) y las variables locales que esa instancia de la función haya creado.,

A medida que se desarrolla la cascada de invocaciones recursivas, el stack segment se va llenando para darle soporte a cada instancia recursiva, y luego, cuando una instancia logra finalizar y se produce el proceso de vuelta atrás, el stack segment comienza a vaciarse.



Historial de respuestas				
Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:28:38	Guardada: A medida que se desarrolla la cascada de invocaciones recursivas, el stack segment se va llenando para darle soporte a cada instancia recursiva, y luego, cuando una instancia logra finalizar y se produce el proceso de vuelta atrás, el stack segment comienza a vaciarse.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Parcialmente correcta	1

Pregunta **3**

Incorrecta

Se puntúa 0 sobre 1

v1 (última)

Suponga que se tiene una cola c en la cual se almacenaron ya una cierta cantidad de datos. ¿Cuál de las siguientes secuencias de instrucciones permite *invertir* la cola? (o sea: si la cola original c era: [3 - 4 - 6 - 7] (con el 3 al frente), ¿cuál de las siguientes permitiría dejar la cola c en el estado [7 - 6 - 4 - 3] (con el 7 al frente)?)

Seleccione una:

- ☐ a.

```
# suponga que c2 y c3 son dos colas inicialmente vacías, y listas para usar
while not c.is_empty():
    c2.add(c2, c.remove())
while not c2.is_empty():
    c2.add(c2.remove())
while not c3.is_empty():
    c.add(c3.remove())
```
- ☐ b.

```
# suponga que p es una PILA vacía y lista para usar...
while not c.is_empty():
    p.push(c.remove())
while not p.is_empty():
    c.add(p.pop())
```
- ☐ c.

```
# suponga que p1 y p1 son dos PILAS inicialmente vacías, y listas para usar
while not c.is_empty():
    p1.push(c.remove())
while not p1.is_empty():
    p2.push(p1.pop())
while not p2.is_empty():
    c.add(p2.pop())
```
- ☒ d.

```
# suponga que c2 es OTRA cola vacía y lista para
usar...
while not c.is_empty():
    c2.add(c.remove())
while not c2.is_empty():
    c.add(c2.remove())
```

✗ Incorrecto... esta secuencia no logra invertir la cola original...

La respuesta correcta es:

```
# suponga que p es una PILA vacía y lista para usar...
while not c.is_empty():
    p.push(c.remove())
while not p.is_empty():
    c.add(p.pop())
```

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:32:20	Guardada: # suponga que p es una PILA vacía y lista para usar... while not c.is_empty(): p.push(c.remove()) while not p.is_empty(): c.add(p.pop())	Respuesta guardada	

Paso	Hora	Acción	Estado	Puntos
3	25/07/24, 08:54:01	Guardada: # suponga que c2 es OTRA cola vacía y lista para usar... while not c.is_empty(): c2.add(c.remove()) while not c2.is_empty(): c.add(c2.remove())	Respuesta guardada	
4	25/07/24, 08:54:22	Intento finalizado	Incorrecta	0

Pregunta 4

Correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuántas comparaciones en el **peor caso** obliga a hacer una búsqueda secuencial en una *lista ordenada* (o en un *arreglo ordenado*) que contenga n valores?

Seleccione una:

- ☒ a. Peor caso: $O(n)$ ✓ ¡Correcto! Aún estando ordenado el arreglo, en el peor caso una búsqueda secuencial deberá llegar hasta el final si el valor buscado no existe en ese arreglo (o existe y está muy atrás)
- ☐ b. Peor caso: $O(n^2)$ comparaciones.
- ☐ c. Peor caso: $O(1)$ comparaciones.
- ☐ d. Peor caso: $O(\log(n))$ comparaciones.

¡Correcto!

La respuesta correcta es:

Peor caso: $O(n)$ comparaciones.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:32:34	Guardada: Peor caso: $O(n)$ comparaciones.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **5**

Incorrecta

Se puntúa 0 sobre 1

v1 (última)

¿Cuál es la principal característica de todos los métodos de ordenamiento conocidos como métodos simples o directos?

Seleccione una:

- ☒ a. Tienen muy mal rendimiento en tiempo de ejecución, cualquiera sea el tamaño n del arreglo. **Incorrecto... Si n es pequeño, estos métodos tienen una demora aceptable...**
- ☐ b. Tienen muy mal rendimiento en tiempo de ejecución si el tamaño n del arreglo es grande o muy grande, y un rendimiento aceptable si n es pequeño.
- ☐ c. Tienen muy buen rendimiento en tiempo de ejecución, cualquiera sea el tamaño n del arreglo.
- ☐ d. Tienen muy mal rendimiento en tiempo de ejecución si el tamaño n del arreglo es pequeño, y un rendimiento aceptable si n es grande o muy grande.

La respuesta correcta es:

Tienen muy mal rendimiento en tiempo de ejecución si el tamaño n del arreglo es grande o muy grande, y un rendimiento aceptable si n es pequeño.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:33:10	Guardada: Tienen muy mal rendimiento en tiempo de ejecución, cualquiera sea el tamaño n del arreglo.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Incorrecta	0

Pregunta 6

Correcta

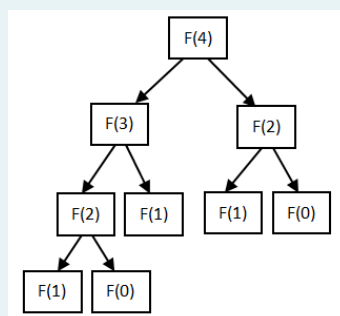
Se puntúa 1 sobre 1

v1 (última)

Sabemos que la recursión es una de las técnicas o estrategias básicas para el planteo de algoritmos y que una de sus ventajas es que permite el diseño de algoritmos compactos y muy claros, aunque al costo de usar algo de memoria extra en el stack segment y por consiguiente también un algo de tiempo extra por la gestión del stack. Algunos problemas pueden plantearse en forma directa procesando recursivamente diversos subproblemas menores. Tal es el caso de la *sucesión de Fibonacci*, en la cual cada término se calcula como la suma de los dos inmediatamente anteriores [esto se expresa con la siguiente relación de recurrencia: $F(n) = F(n-1) + F(n-2)$ con $F(0) = 0$ y $F(1) = 1$]. La función sencilla que mostramos a continuación que calcula el término n-ésimo de la sucesión en forma recursiva:

```
def fibo(n):
    if n <= 1:
        return 1
    return fibo(n-1) + fibo(n-2)
```

Sin embargo, un inconveniente adicional es que la *aplicación directa* de *dos o más invocaciones recursivas* en el planteo de un algoritmo podría hacer que un mismo subproblema se resuelva más de una vez, incrementando el tiempo total de ejecución. Por ejemplo, para calcular $\text{fibo}(4)$ el árbol de llamadas recursivas es el siguiente:



y puede verse que en este caso, se calcula **2 veces** el valor de ***fibo(2)***, **3 veces** el valor de ***fibo(1)*** y **2 veces** el valor de ***fibo(0)***... con lo que la cantidad de llamadas a funciones para hacer *más de una vez el mismo trabajo* es de **7** ($= 2 + 3 + 2$).

Suponga que se quiere calcular el valor del sexto término de la sucesión. **Analice el árbol de llamadas recursivas que se genera al hacer la invocación $t = \text{fibo}(6)$ ¿Cuántas veces en total la función *fibo()* se invoca para hacer más de una vez el mismo trabajo, SIN incluir en ese conteo a las invocaciones para obtener *fibo(0)* y *fibo(1)*?**

Seleccione una:

- ☐ a. 11
- ☐ b. 12
- ☒ c. 10 **✓ ¡Correcto! Efectivamente: *fibo(4)* se invoca 2 veces, *fibo(3)* se invoca 3 veces y *fibo(2)* se invoca 5 veces...**
- ☐ d. 0

¡Correcto!

La respuesta correcta es: 10

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:33:25	Guardada: 10	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **7**

Parcialmente correcta

Se puntúa 1 sobre 1

v1 (última)

En la columna de la izquierda se muestra el código fuente en Python de diversos procesos sencillos. Para cada uno de ellos, seleccione de la lista de la derecha la expresión en notación *Big O* que mejor describa el tiempo de ejecución de cada proceso para el peor caso. (Aclaración: una expresión como n^2 debe entenderse como "*n al cuadrado*" o n^2).

```
n = int(input('N: '))
ac = 0
for i in range(n):
    ac += i
print(ac)
```

O(n)



```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j

for i in range(n):
    for j in range(n):
        for k in range(n):
            ac += (i+j+k)
print(ac)
```

O(n^2)



```
ac = 0
for i in range(10):
    ac += i
print(ac)
```

O(n^3)



```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j
print(ac)
```

O(n^2)



Ha seleccionado correctamente 2.

La respuesta correcta es:

```
n = int(input('N: '))
ac = 0
for i in range(n):
    ac += i
print(ac)
```

→ O(n),

```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j

for i in range(n):
    for j in range(n):
        for k in range(n):
            ac += (i+j+k)
print(ac)
```

→ O(n^3),

```
ac = 0
for i in range(10):
    ac += i
print(ac)
```

→ O(1),

```
n = int(input('N: '))
ac = 0
for i in range(n):
    for j in range(i+1, n):
        ac += i*j
print(ac)
```

→ $O(n^2)$

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:36:43	Guardada: n = int(input('N: ')) ac = 0 for i in range(n): ac += i print(ac) -> $O(n)$; n = int(input('N: ')) ac = 0 for i in range(n): for j in range(i+1, n): ac += i*j for i in range(n): for j in range(n): for k in range(n): ac += (i+j+k) print(ac) -> $O(n^2)$; ac = 0 for i in range(10): ac += i print(ac) -> $O(n^3)$; n = int(input('N: ')) ac = 0 for i in range(n): for j in range(i+1, n): ac += i*j print(ac) -> $O(n^2)$	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Parcialmente correcta	1

Pregunta **8**

Parcialmente correcta

Se puntúa 1 sobre 1

v1 (última)

Considere el problema de las *Ocho Reinas* presentado en clases. ¿Cuáles de las siguientes afirmaciones son **ciertas** en relación a las **diagonales del tablero** en el cual deben colocarse la reinas, suponiendo que el tablero es el normal del ajedrez, de 8 * 8? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☐ a. En cada una de las diagonales que se orientan como la principal, es constante la resta entre el número de columna y el número de fila de cada uno de sus elementos.
- ☒ b. En general hay dos tipos de diagonales: las normales (orientadas en la misma forma que la diagonal principal) y las inversas (orientadas en la misma forma que la contra-diagonal o diagonal inversa). ¡Correcto!
- ☒ c. La cantidad **total** de diagonales que contiene el tablero (sumando todas las diagonales de todos los tipos posibles) es 30. ¡Correcto! Efectivamente... son 15 normales y 15 inversas...
- ☐ d. En cada una de las diagonales que se orientan como la contra-diagonal o diagonal inversa, es constante la suma entre el número de columna y el número de fila de cada uno de sus elementos.

Las respuestas correctas son:

En general hay dos tipos de diagonales: las normales (orientadas en la misma forma que la diagonal principal) y las inversas (orientadas en la misma forma que la contra-diagonal o diagonal inversa).,

La cantidad **total** de diagonales que contiene el tablero (sumando todas las diagonales de todos los tipos posibles) es 30.,

En cada una de las diagonales que se orientan como la principal, es constante la resta entre el número de columna y el número de fila de cada uno de sus elementos.,

En cada una de las diagonales que se orientan como la contra-diagonal o diagonal inversa, es constante la suma entre el número de columna y el número de fila de cada uno de sus elementos.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:39:31	Guardada: En general hay dos tipos de diagonales: las normales (orientadas en la misma forma que la diagonal principal) y las inversas (orientadas en la misma forma que la contra-diagonal o diagonal inversa). ; La cantidad _TOTAL_ de diagonales que contiene el tablero (sumando todas las diagonales de todos los tipos posibles) es 30.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Parcialmente correcta	1

Pregunta 9

Correcta

Se puntúa 1 sobre 1

v1 (última)

Una *Cola de Prioridad* es una cola en la cual los elementos se insertan en algún orden, pero tal que cuando se pide retirar un elemento se obtiene siempre *el menor* de los valores almacenados en la cola ¿Cuál de las siguientes estrategias podría ser una forma básica de implementar una *Cola de Prioridad* en las condiciones aquí expresadas?

Seleccione una:

- ☒ a. Soporte: una variable de tipo *list* en Python. Inserción: *ordenada de menor a mayor* (mantener el arreglo siempre ordenado de menor a mayor: para insertar un valor *x*, recorrer el arreglo y al encontrar el primer mayor a *x* detener el ciclo y añadir allí a *x* con un corte de índices. Eliminación: siempre el primer elemento.
- ☐ b. Soporte: una variable de tipo *list* en Python. Inserción: siempre al frente. Eliminación: siempre el primer elemento.
- ☐ c. Soporte: una variable de tipo *list* en Python. Inserción: *ordenada de mayor a menor* (mantener el arreglo siempre ordenado de mayor a menor: para insertar un nuevo valor *x*, recorrer el arreglo y al encontrar el primer menor, a *x* detener el ciclo y añadir allí a *x* con un corte de índices. Eliminación: siempre el primer elemento.
- ☐ d. Soporte: una variable de tipo *list* en Python. Inserción: siempre al final. Eliminación: siempre el último elemento.

¡Correcto! Aunque entienda que si se hace así, logrará el objetivo pero la inserción tendrá un tiempo de ejecución $O(n)$ mientras que la eliminación será de tiempo constante ($O(1)$).

¡Correcto!

La respuesta correcta es:

Soporte: una variable de tipo *list* en Python. Inserción: *ordenada de menor a mayor* (mantener el arreglo siempre ordenado de menor a mayor: para insertar un valor *x*, recorrer el arreglo y al encontrar el primer mayor a *x* detener el ciclo y añadir allí a *x* con un corte de índices. Eliminación: siempre el primer elemento.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:42:11	Guardada: Soporte: una variable de tipo <code>_list_</code> en Python. Inserción: <code>_ordenada de menor a mayor_</code> (mantener el arreglo siempre ordenado de menor a mayor: para insertar un valor <code>_x_</code> , recorrer el arreglo y al encontrar el primer mayor a <code>_x_</code> detener el ciclo y añadir allí a <code>_x_</code> con un corte de índices. Eliminación: siempre el primer elemento.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **10**

Correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuáles de las siguientes afirmaciones **son ciertas** en referencia a las *Estrategias de Resolución de Problemas* que se citan? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado. ✓ ¡Correcto!
- ☐ b. El empleo de la *Recursividad* para resolver un problema no es recomendable en ningún caso, debido a la gran cantidad de recursos de memoria o de tiempo de ejecución que implica.
- ☒ c. La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción. ✓ ¡Correcto!
- ☒ d. La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor. ✓ ¡Correcto!

¡Correcto!

Las respuestas correctas son:

La estrategia de *Fuerza Bruta* se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado.,

La estrategia de *Backtracking* es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción.,

La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:43:06	Guardada: La estrategia de <i>_Fuerza Bruta_</i> se basa en aplicar ideas intuitivas y directas, simples de codificar, pero normalmente produce algoritmos de mal rendimiento en tiempo de ejecución y/o de espacio de memoria empleado. ; La estrategia de <i>_ Backtracking_</i> es de base recursiva y permite implementar soluciones de prueba y error explorando las distintas soluciones y volviendo atrás si se detecta que un camino conduce a una solución incorrecta. cuando es aplicable, es más eficiente que la Fuerza Bruta, ya que permite eliminar caminos por deducción. ; La técnica de Programación Dinámica se basa en calcular los resultados de los subproblemas de menor orden o tamaño que pudieran aparecer, almacenar esos resultados en una tabla, y luego re-usarlos cuando vuelvan a ser requeridos en el cálculo del problema mayor.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **11**

Correcta

Se puntúa 1 sobre 1

v1 (última)

Suponga que para un problema dado se ha planteado un algoritmo basado en divide y vencerás cuya estructura lógica esencial es la siguiente:

```
proceso(partición):  
  adicional() [O(n)]  
  proceso(partición/3)  
  proceso(partición/3)  
  proceso(partición/3)
```

¿Cuál de las siguientes es la expresión en *notación Big O* del tiempo de ejecución para este proceso? (Aclaración: la base del logaritmo no es esencial en una expresión en notación Big O, pero en este caso es relevante a los efectos del planteo conceptual de la pregunta).

Seleccione una:

- ☐ a. $O(n^2 * \log_3(n))$
- ☐ b. $O(n^2 * \log_2(n))$
- ☒ c. $O(n * \log_3(n))$ ✓ ¡Correcto!
- ☐ d. $O(n * \log_2(n))$

¡Correcto!

La respuesta correcta es:
 $O(n * \log_3(n))$



Historial de respuestas				
Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:43:34	Guardada: $O(n * \log_3(n))$	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **12**

Correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuál de las siguientes situaciones haría que el algoritmo *Quicksort* degenera en su peor caso en cuanto al tiempo de ejecución, de orden n^2 ?

Seleccione una:

- ☐ a. Que el arreglo esté ya ordenado, pero al revés.
- ☒ b. Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando. ✓ ¡Correcto!
- ☐ c. El algoritmo Quicksort no tiene un peor caso $O(n^2)$. Su tiempo de ejecución siempre es $O(n \cdot \log(n))$.
- ☐ d. Que el arreglo esté ya ordenado, en la misma secuencia en que se lo quiere ordenar.

¡Correcto!

La respuesta correcta es:

Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:44:12	Guardada: Que el arreglo de entrada tenga sus elementos dispuestos de tal forma que cada vez que se seleccione el pivot en cada partición, resulte que ese pivot sea siempre el menor o el mayor de la partición que se está procesando.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **13**

Parcialmente correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuáles de las siguientes son características **correctas** del algoritmo *Shellsort*? (Más de una puede ser cierta... marque TODAS las que considere válidas)

Seleccione una o más de una:

- ☒ a. Una muy buena serie de incrementos decrecientes a usar, **✗** **Incorrecto... en esa serie, todos los valores son múltiplos entre sí... y eso es una mala idea...**
- ☒ b. El algoritmo Shellsort consiste en una mejora del algoritmo de Inserción Directa (o Inserción Simple), consistente **✓** **¡Correcto!** en armar subconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última.
- ☒ c. El algoritmo Shellsort consiste en una mejora del algoritmo de Selección Directa, basada en buscar **✗** **Incorrecto... esa es la idea del Heap Sort (y no la del Shell Sort...)** iterativamente el menor (o el mayor) entre los elementos que quedan en el vector, para llevarlo a su posición correcta, pero de forma que la búsqueda del menor en cada vuelta se haga en tiempo logarítmico.
- ☐ d. El algoritmo Shellsort es complejo de analizar para determinar su rendimientos en forma matemática, ya que ese rendimiento depende fuertemente de la serie de incrementos decreciente que se haya seleccionado.

Las respuestas correctas son:

El algoritmo Shellsort consiste en una mejora del algoritmo de Inserción Directa (o Inserción Simple), consistente en armar subconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última.,

El algoritmo Shellsort es complejo de analizar para determinar su rendimientos en forma matemática, ya que ese rendimiento depende fuertemente de la serie de incrementos decreciente que se haya seleccionado.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:46:06	Guardada: Una muy buena serie de incrementos decrecientes a usar, es la serie $h = \{...16, 8, 4, 2, 1\}$; El algoritmo Shellsort consiste en una mejora del algoritmo de Inserción Directa (o Inserción Simple), consistente en armar subconjuntos ordenados con elementos a distancia $h > 1$ en las primeras fases, y terminar con $h = 1$ en la última. ; El algoritmo Shellsort consiste en una mejora del algoritmo de Selección Directa, basada en buscar iterativamente el menor (o el mayor) entre los elementos que quedan en el vector, para llevarlo a su posición correcta, pero de forma que la búsqueda del menor en cada vuelta se haga en tiempo logarítmico.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Parcialmente correcta	1

Pregunta **14**

Correcta

Se puntúa 1 sobre 1

v1 (última)

Si los algoritmos de *ordenamiento simples* tienen todos un tiempo de ejecución $O(n^2)$ en el peor caso, entonces: ¿cómo explica que las mediciones efectivas de los tiempos de ejecución de cada uno sean diferentes frente al mismo arreglo?

Seleccione una:

- ☒ a. La notación *Big O* rescata el término más significativo en la expresión que calcula el rendimiento, descartando ✓ **¡Correcto!** constantes y otros términos que podrían no coincidir en los tres algoritmos.
- ☐ b. Los tiempos deben coincidir. Si hay diferencias, se debe a errores en los instrumentos de medición o a un planteo incorrecto del proceso de medición.
- ☐ c. La notación *Big O* no se debe usar para estimar el comportamiento en el peor caso, sino sólo para el caso medio.
- ☐ d. La notación *Big O* no se usa para medir tiempos sino para contar comparaciones u otro elemento de interés. Es un error, entonces, decir que los tiempos tienen "*orden n cuadrado*".

¡Correcto!

La respuesta correcta es:

La notación *Big O* rescata el término más significativo en la expresión que calcula el rendimiento, descartando constantes y otros términos que podrían no coincidir en los tres algoritmos.

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:46:41	Guardada: La notación <i>_Big O_</i> rescata el término más significativo en la expresión que calcula el rendimiento, descartando constantes y otros términos que podrían no coincidir en los tres algoritmos.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **15**

Correcta

Se puntúa 1 sobre 1

v1 (última)

Considere la función presentada en clases para calcular mediana entre el primero, el central y el último elemento de una partición del arreglo v delimitada por los elementos en las posiciones izq y der :

```
def get_pivot_m3(v, izq, der):  
    # calculo del pivot: mediana de tres...  
    central = int((izq + der) / 2)  
  
    if v[der] < v[izq]:  
        v[der], v[izq] = v[izq], v[der]  
  
    if v[central] < v[izq]:  
        v[central], v[izq] = v[izq], v[central]  
  
    if v[central] > v[der]:  
        v[central], v[der] = v[der], v[central]  
  
    return v[central]
```

El tamaño de la partición analizada es entonces $n = der - izq + 1$ elementos. ¿Cuál es el tiempo de ejecución de esta función, expresado en notación O y de acuerdo a ese valor de n ?

Seleccione una:

- ☐ a. $O(n^2)$
- ☐ b. $O(\log(n))$
- ☒ c. $O(1)$ **✓** ¡Correcto! Efectivamente, el cálculo no depende del tamaño de la partición, por lo que resulta de tiempo constante.
- ☐ d. $O(n)$

¡Correcto!

La respuesta correcta es:
 $O(1)$



Historial de respuestas				
Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:47:37	Guardada: $O(1)$	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **16**

Parcialmente correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuáles de los siguientes son *factores de eficiencia comunes* a considerar en el análisis de algoritmos? (Más de una respuesta puede ser válida... marque *todas* las que considere correctas).

Seleccione una o más de una:

☒ a. El consumo de memoria. ✓ ¡Correcto!

☐ b. La calidad aparente de la interfaz de usuario.

☒ c. El tiempo de ejecución. ✓ ¡Correcto!

☐ d. La complejidad aparente del código fuente.

Las respuestas correctas son:

El tiempo de ejecución.,

El consumo de memoria.,

La complejidad aparente del código fuente.



Historial de respuestas				
Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:48:31	Guardada: El consumo de memoria. ; El tiempo de ejecución.	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Parcialmente correcta	1

Pregunta **17**

Correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuál de los siguientes es el creador del famoso algoritmo de ordenamiento conocido como **Quicksort**?

Seleccione una:

☒ a.

 Charles Antony Richard Hoare ✓ ¡Correcto!

☐ b.

 Donald Shell☐ c.☐ d.

¡Correcto!

La respuesta correcta es:

Charles Antony Richard Hoare



Historial de respuestas				
Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:48:44	Guardada: Charles Antony Richard Hoare	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **18**

Correcta

Se puntúa 1 sobre 1

v1 (última)

Considere el problema de las *Ocho Reinas* presentado en clases. Se ha indicado que se puede usar un arreglo rc de componentes, en el cual el casillero $rc[col] = fil$ indica que la reina de la columna col está ubicada en la fila fil . ¿Cuáles de las siguientes configuraciones para el arreglo rc representan ***soluciones incorrectas*** para el problema de las *Ocho Reinas*? (Más de una respuesta puede ser cierta, por lo que marque todas las que considere correctas...)

Seleccione una o más de una:

- ☒ a. $rc = [3, 5, 7, 0, 5, 1, 2, 4]$ ¡Correcto! Efectivamente, si rc contiene los valores mostrados, hay al menos un problema: las reinas de las columnas 1 y 4 están ubicadas en la misma fila: la 5.
- ☒ b. $rc = [2, 0, 7, 4, 5, 1, 6, 3]$ ¡Correcto! Efectivamente, si rc contiene los valores mostrados, **hay al menos** un problema: las reinas de las columnas 3 y 4 están ubicadas en la misma diagonal normal: la diagonal -1.
- ☐ c. $rc = [4, 7, 3, 0, 2, 5, 1, 6]$
- ☐ d. $rc = [5, 3, 6, 0, 7, 1, 4, 2]$

¡Correcto!

Las respuestas correctas son:

$rc = [3, 5, 7, 0, 5, 1, 2, 4]$,

$rc = [2, 0, 7, 4, 5, 1, 6, 3]$

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:50:53	Guardada: $rc = [3, 5, 7, 0, 5, 1, 2, 4]$; $_rc_ = [2, 0, 7, 4, 5, 1, 6, 3]$	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **19**

Correcta

Se puntúa 1 sobre 1

v1 (última)

Para cada uno de los algoritmos o procesos listados en la columna de la izquierda, seleccione la expresión de orden que mejor describe su tiempo de ejecución en el peor caso.

Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero v[k] (con $0 \leq k \leq n-1$).	O(1) ▼ ✓
Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial.	O(n) ▼ ✓
Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa.	O(n²) (léase: "orden n al cuadrado") ▼ ✓
Dadas dos matrices de orden n*n, obtener la matriz producto aplicando el método tradicional.	O(n³) (léase: "orden n al cubo") ▼ ✓
Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria.	O(log(n)) ▼ ✓

¡Correcto!

La respuesta correcta es:

Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero v[k] (con $0 \leq k \leq n-1$). → O(1),

Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial. → O(n),

Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa.

→ O(n²) (léase: "orden n al cuadrado"),

Dadas dos matrices de orden n*n, obtener la matriz producto aplicando el método tradicional. → O(n³) (léase: "orden n al cubo"),

Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria.

→ O(log(n))

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:51:55	Guardada: Dado un arreglo v con n componentes, acceder y cambiar el valor del casillero v[k] (con $0 \leq k \leq n-1$). -> O(1); Dado un arreglo v con n componentes, buscar un valor x aplicando búsqueda secuencial. -> O(n); Dado un arreglo v con n componentes, ordenarlo de menor a mayor mediante el algoritmo de selección directa. -> O(n ²) (léase: "orden n al cuadrado"); Dadas dos matrices de orden n*n, obtener la matriz producto aplicando el método tradicional. -> O(n ³) (léase: "orden n al cubo"); Dado un arreglo ya ordenado v con n componentes, buscar un valor x aplicando búsqueda binaria. -> O(log(n))	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

Pregunta **20**

Correcta

Se puntúa 1 sobre 1

v1 (última)

¿Cuál de los siguientes esquemas simples de pseudocódigo representa a un algoritmo planteado mediante estrategia Divide y Vencerás, pero con tiempo de ejecución $O(n \log(n))$?

Seleccione una:

- ☐ a. proceso(partición):
 adicional() [$\Rightarrow O(n^2)$]
 proceso(partición/2)
 proceso(partición/2)
- ☐ b. proceso(partición):
 adicional() [$\Rightarrow O(n^3)$]
 proceso(partición/2)
 proceso(partición/2)
- ☒ c. proceso(partición): ✓ ¡Correcto!
 adicional() [$\Rightarrow O(n)$]
 proceso(partición/2)
 proceso(partición/2)
- ☐ d. proceso(partición):
 adicional() [$\Rightarrow O(1)$]
 proceso(partición/2)
 proceso(partición/2)

¡Correcto!

La respuesta correcta es:

```
proceso(partición):  
    adicional() [ $\Rightarrow O(n)$ ]  
    proceso(partición/2)  
    proceso(partición/2)
```

Historial de respuestas

Paso	Hora	Acción	Estado	Puntos
1	25/07/24, 08:25:35	Iniciado/a	Sin responder aún	
2	25/07/24, 08:52:23	Guardada: proceso(partición): adicional() [$\Rightarrow O(n)$] proceso(partición/2)	Respuesta guardada	
3	25/07/24, 08:54:22	Intento finalizado	Correcta	1

◀ Cuestionario Teórico (Regulares y Promocionados - 2023)

Ir a...



Enunciado Examen Final Práctico (Regulares - Todos los años) ▶